

# **BÀI TẬP C# – OOP & COLLECTIONS**

Giảng viên: Phieu Tu

Ngày 2 tháng 2 năm 2026

## **Mục lục**

|          |                                         |          |
|----------|-----------------------------------------|----------|
| <b>1</b> | <b>Bài 1: Library Management System</b> | <b>2</b> |
| <b>2</b> | <b>Bài 2: Student Management</b>        | <b>2</b> |
| <b>3</b> | <b>Bài 3: Product Inventory</b>         | <b>3</b> |
| <b>4</b> | <b>Bài 4: Bank Account</b>              | <b>4</b> |
| <b>5</b> | <b>Bài 5: Game Character</b>            | <b>4</b> |

## 1 Bài 1: Library Management System

### Mô tả bài tập

Xây dựng hệ thống quản lý thư viện với class `Book` và `List<Book>`.

### Yêu cầu

1. Tạo class `Book` gồm: `Title`, `Author`, `ISBN`, `YearPublished`
2. Viết constructor đầy đủ tham số
3. Tạo `List<Book>` chứa ít nhất 3 cuốn sách
4. In thông tin từng sách bằng `foreach`

### Câu hỏi 01

- Q1. Thêm validation cho `YearPublished` (1000 → năm hiện tại)
- Q2. Viết constructor overload (không tham số, 2 tham số)
- Q3. Override `Equals()` để so sánh sách theo `ISBN`

### Câu hỏi 02

- A1. Implement `IComparable<Book>` để sắp xếp sách theo năm xuất bản
- A2. Viết method `GetBooksAfterYear(int year)` (không dùng LINQ)
- A3. Đếm số sách theo tác giả và lưu vào `Dictionary<string, int>`

## 2 Bài 2: Student Management

### Mô tả bài tập

Quản lý sinh viên bằng `Dictionary<string, Student>`.

### Yêu cầu

1. Class `Student`: `Id`, `Name`, `Grade`
2. Dictionary với `key = Name`, `value = Student`
3. Thêm 3 sinh viên và in thông tin

**Câu hỏi 01**

- Q1. Viết method `AddStudentSafe()`
- Q2. Tìm sinh viên có điểm cao nhất (dùng vòng lặp)
- Q3. Chuyển Dictionary sang `List<Student>`

**Câu hỏi 02**

- A1. Tạo generic class `Repository<TKey, TValue>`
- A2. Implement indexer truy cập Student theo Name
- A3. Viết method in sinh viên có `Grade ≥` giá trị cho trước

### 3 Bài 3: Product Inventory

**Mô tả bài tập**

Quản lý kho sản phẩm bằng `List<Product>`.

**Yêu cầu**

- 1. Class Product: Id, Name, Price, Quantity
- 2. Method `GetTotalValue()`
- 3. Tạo List chứa 5 sản phẩm

**Câu hỏi 01**

- Q1. Tìm sản phẩm tồn kho thấp hơn ngưỡng cho trước
- Q2. Tính tổng giá trị kho
- Q3. Tìm sản phẩm có giá trị cao nhất

**Câu hỏi 02**

- A1. Tạo class Inventory implement `IEnumerable<Product>`
- A2. Viết method `UpdateStock()`
- A3. Sử dụng foreach để duyệt Inventory

## 4 Bài 4: Bank Account

### Mô tả bài tập

Xây dựng hệ thống tài khoản ngân hàng sử dụng `List<Transaction>`.

### Yêu cầu

1. Class `BankAccount`: `AccountNumber`, `Holder`, `Balance`
2. Class `Transaction`: `Type`, `Amount`, `Date`
3. `Deposit`, `Withdraw`, In lịch sử giao dịch

### Câu hỏi 01

- Q1. Implement method `Transfer()`
- Q2. Kiểm tra số dư hợp lệ khi rút
- Q3. Sử dụng enum `AccountType`

### Câu hỏi 02

- A1. Refactor bằng abstract class `BankAccount`
- A2. Override `Withdraw()` cho từng loại tài khoản
- A3. Lưu lịch sử giao dịch bằng `List`

## 5 Bài 5: Game Character

### Mô tả bài tập

Quản lý nhân vật game với `List` và `Dictionary`.

### Yêu cầu

1. Class `Character`: `Name`, `Level`, `Health`
2. `Inventory`: `Dictionary<string,int>`
3. `Skills`: `List<string>`

### Câu hỏi 01

- Q1. Kiểm tra item trong inventory
- Q2. Không cho học trùng skill
- Q3. `LevelUp` tăng `Level` và `Health`

**Câu hỏi 02**

- A1. Implement `Comparer<Character>` (Level giảm dần)
- A2. Deep copy List và Dictionary
- A3. So sánh hai Character

— HẾT —